

4. Fases del desarrollo del software

*Análisis · Diseño · Implementación · Pruebas · Despliegue · Operación · Documentación ·
Modelos de ciclo de vida*

Visión general del ciclo

Desarrollar software no es solo programar: antes hay que entender qué se necesita, pensar cómo organizarlo, implementarlo, probarlo, entregarlo y mantenerlo. Cada actividad es una FASE, y la salida de una es la entrada de la siguiente.

¿Por qué existen las fases? Como construir una casa: si pones ladrillos sin plano, cuando llegue el fontanero la tubería ya no cabe. Cada euro que cuesta un error en producción habría costado mucho menos detectado antes. Las fases detectan problemas cuando aún son baratos.

Fase	Pregunta clave	Resultado típico
4.1 Análisis	¿Qué debe hacer el sistema?	Lista de requisitos, historias de usuario
4.2 Diseño	¿Cómo lo organizamos?	Diagramas, decisiones de arquitectura
4.3 Implementación	¿Cómo lo codificamos?	Código fuente revisado
4.4 Pruebas	¿Funciona como se espera?	Informes de prueba, bugs corregidos
4.5 Despliegue	¿Cómo lo entregamos?	Versión publicada
4.6 Operación	¿Sigue funcionando?	Métricas, alertas, parches
4.7 Documentación	¿Cómo lo explicamos?	README, guías, comentarios

4.1 Análisis de requisitos

El análisis transforma una necesidad en una lista de condiciones VERIFICABLES que el software debe cumplir. Participan los stakeholders — las personas interesadas: cliente, usuarios finales, equipo técnico, dirección — con el objetivo de que todos acaben con la misma imagen de lo que hay que construir.

Tipo de requisito	Qué describe	Ejemplo
Funcional	QUÉ debe hacer el sistema: pantallas, flujos, validaciones, reglas de negocio	"El sistema permite registrar usuarios con email y contraseña"
No funcional	CÓMO debe ser: rendimiento, seguridad, accesibilidad, cumplimiento legal	"El inicio de sesión responde en menos de 300 ms para el 95% de las peticiones"

Si esta fase queda poco clara, los problemas aparecen en la implementación o — peor — cuando el programa ya está en manos de los usuarios.

4.1 Historias de usuario y criterios INVEST

Una historia de usuario captura una necesidad desde el punto de vista de quien usará el sistema. Se usa en entornos ágiles para planificar y priorizar:

Historia: Recuperación de contraseña
Como cliente
Quiero restablecer mi contraseña
Para recuperar acceso si la olvido

Criterios de aceptación

(Given/When/Then) :

Given estoy en "Olvidé mi contraseña"
When introduzco mi email y confirmo
Then recibo un enlace de recuperación
válido durante 30 minutos

Letra	Significa	Qué implica
I	Independiente	No depende de otra historia
N	Negociable	El cómo (y el qué) puede cambiar si hay razón
V	Valiosa	Aporta algo concreto al usuario o negocio
E	Estimable	El equipo puede calcular cuánto cuesta
S	Simple	Cabe en una iteración; si no, se parte
T	Testable	Criterios de aceptación verificables

A cada historia se le añaden criterios de aceptación: condiciones concretas que deben cumplirse para considerarla terminada. Una buena historia cumple los seis criterios INVEST.

4.2 Diseño: arquitectura, patrones y diagramas UML

El diseño es el puente entre lo que queremos (requisitos) y cómo lo construiremos (código): divide el problema en piezas, acuerda cómo encajan y evita sorpresas al programar.

Arquitectura	Cómo se organiza	Cuándo usarla
Monolito	Todo el código en una sola aplicación. Fácil de arrancar, entender y desplegar.	Proyectos pequeños, primeras prácticas, equipos de 1-2 personas
Tres capas	Presentación (pantallas) → Lógica (reglas y cálculos) → Datos (BD/archivos)	Apps web o escritorio de tamaño medio — la más habitual en DAW y DAM
Servicios	Piezas independientes que se comunican por mensajes. Escala bien, pero añade complejidad.	Sistemas grandes con equipos independientes; no para empezar

Diagramas UML — solo los más útiles

Diagrama	Qué muestra
Clases	Qué datos hay y cómo se relacionan (ej.: Carrito contiene Productos, con sus métodos)
Actividad	Los pasos de un flujo: inicio → decisiones → fin (ej.: login con reintento si la contraseña falla)
Casos de uso	Quién hace qué: una lista de acciones y actores ya basta — "Registrar (Cliente)", "Ver pedidos (Admin)"

4.3 Implementación — principios básicos

Objetivo: convertir el diseño en código que funciona y que CUALQUIER COMPAÑERO pueda leer y mantener. No es solo "que compile": importa que sea claro, coherente y fácil de cambiar.

Principio	Qué significa	Ejemplo
Claridad	Escribe para quien lo leerá después (muchas veces tú en 6 meses). Los nombres explican qué hacen sin comentarios.	<code>double totalConImpuestos = calcularTotalPedido(precio, iva)</code> — no <code>double r = f(p, i)</code>
Una responsabilidad	Cada función hace UNA sola cosa. Si valida + calcula + envía emails, es difícil de probar y cambiar.	Separar en <code>validar(p)</code> , <code>calcularTotal(p)</code> , <code>guardar(p)</code> , <code>notificarCliente(p)</code>
DRY (no te repitas)	Si copias y pegas el mismo bloque en varios sitios, al cambiarlo tendrás que buscarlo en todos.	Extraer el bloque repetido a una función reutilizable
Comentarios útiles	No comentes lo que el código ya dice; comenta el PORQUÉ: decisiones, reglas de negocio, restricciones.	<code>// Si b es 0 devolvemos 0 por política de negocio (evita dividir por cero)</code>
Estilo consistente	Misma indentación, llaves y espacios en todo el proyecto.	Un formateador automático (Checkstyle en Java) evita discusiones y errores

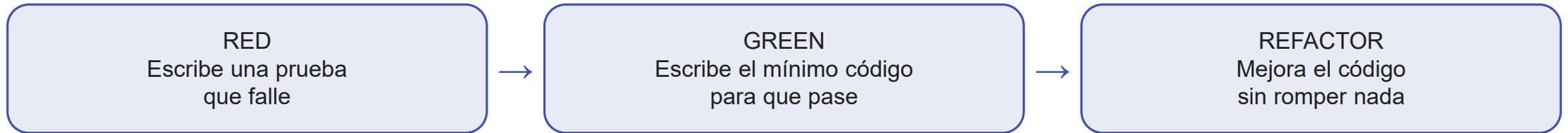
4.4 Pruebas — la pirámide y los tipos

Probar no es desconfiar del código: es aprender rápido si algo funciona como esperamos. Cuanto antes pruebas, más barato es corregir. La pirámide: muchas pruebas rápidas y baratas en la base, pocas lentas y costosas arriba.

Tipo	Qué prueba	Ejemplo	Herramientas
Unitarias (muchas)	Una sola función o clase aislada. Las más rápidas y fáciles de mantener.	calcularTotal(10, 0.21) devuelve 12.1	JUnit (Java), Jest (JS), pytest (Python)
Integración (algunas)	El encaje entre piezas: función + BD, servicio + API externa.	Guardar un pedido en la BD de pruebas y leerlo después	BD de pruebas, mocks
Sistema / E2E (pocas)	El flujo completo tal como lo ve el usuario.	Login → añadir al carrito → pagar → confirmación	Selenium, Cypress, Playwright
Aceptación	Que el sistema cumple los requisitos acordados con el cliente.	Basadas en los criterios de aceptación de las historias	Given/When/Then

4.4 Pruebas — TDD y BDD

TDD (Test-Driven Development): la prueba ANTES que el código



Ventaja: al terminar tienes el código Y las pruebas a la vez, y el diseño suele ser más simple porque solo escribes lo que la prueba necesita.

BDD (Behavior-Driven Development): comportamiento en lenguaje común

```
Feature: Recuperar contraseña
  Scenario: Enlace válido
    Given estoy en "Olvidé mi contraseña"
    When introduzco mi email y confirmo
    Then recibo un enlace válido durante 30 minutos
```

Describe el comportamiento esperado en un lenguaje que el equipo técnico Y el cliente entienden. Se usa sobre todo para pruebas de aceptación.

4.5 Despliegue — dónde vive la app y estrategias

Desplegar es llevar el software de "listo en el repositorio" a "en manos de los usuarios", de forma segura y con plan de vuelta atrás.

Modalidad	Dónde se ejecuta	Ventajas	A tener en cuenta
Servidor propio (on-premises)	Máquinas que tu empresa gestiona físicamente	Control total; los datos no salen de la empresa	Tú te encargas de todo: actualizaciones, backups, seguridad
Nube	Servidores de un proveedor (AWS, Azure, Google Cloud)	Pagas lo que usas; escala fácil	Dependes del proveedor; costes pueden subir sin control
		Distribución masiva	Revisión previa de la tienda; puede
Estrategia	Cómo funciona	Ventaja	Cuándo
Blue/Green	Dos entornos idénticos (azul = actual, verde = nuevo); el tráfico cambia de golpe	Vuelta al azul en segundos si algo falla	Cambios grandes con reversión rápida
Canary	La versión nueva va solo a un % pequeño (5%) y se amplía gradualmente	Los errores afectan a muy poca gente	Observar comportamiento real antes del 100%
Rolling	Las instancias del servidor se actualizan una a una	Sin corte de servicio	Sistemas con varias instancias en paralelo

4.5 Despliegue — versionado SemVer

Las versiones llevan tres números: MAJOR.MINOR.PATCH (por ejemplo, 2.4.7).

Número	Cuándo se incrementa	Ejemplo
PATCH	Arreglo de un bug, sin cambios de funcionalidad	1.3.5 → 1.3.6 (arreglaste un bug)
MINOR	Función nueva que no rompe nada	1.3.6 → 1.4.0 (añadiste una función)
MAJOR	Cambio que rompe compatibilidad con versiones anteriores	1.4.0 → 2.0.0 (los usuarios deben adaptar su código)

Antes de desplegar, comprueba siempre

- Los datos de configuración (contraseñas, URLs de base de datos) están en el entorno destino, NO en el código.
- Tienes copia de seguridad de la base de datos si vas a cambiar su estructura.
- Sabes cómo volver atrás si algo sale mal.

4.6 Operación y mantenimiento

En producción el trabajo no termina: hay que vigilar, resolver y mejorar. Objetivo: que el equipo detecte los problemas ANTES que los usuarios. Para ello se recogen señales automáticas:

Señal	Qué es	Clave
Logs	Mensajes de texto que la app escribe mientras funciona. El primer sitio donde mirar cuando algo falla.	Niveles: DEBUG (desarrollo) · INFO (eventos normales) · WARN (raro pero no roto) · ERROR (hay que actuar)
Métricas	Números medidos a intervalos: peticiones/segundo, tiempo de respuesta, errores/minuto, CPU y memoria.	Se muestran en un dashboard que el equipo consulta en cualquier momento
Alertas	Notificación automática cuando una métrica supera un umbral ("si los errores superan el 5%, avisa").	Deben ser ACCIONABLES: al saltar una, alguien sabe qué hacer. Los runbooks (guías paso a paso por incidencia) permiten actuar sin depender de quien las creó

Mantenimiento	Cuándo	Ejemplo
Correctivo	Bug detectado en producción	Arreglar un error que cierra la app sola
Adaptativo	Cambia el entorno externo	Actualizar la app por una nueva versión de Android
Perfectivo	Mejorar rendimiento o usabilidad	Reducir el tiempo de carga de una pantalla
Preventivo	Evitar problemas futuros	Actualizar dependencias con vulnerabilidades conocidas

4.7 Documentación

La documentación reduce la dependencia de personas clave, acelera el onboarding (la incorporación de una persona nueva al equipo) y baja el coste de mantenimiento. No es un extra: es parte del producto.

Documento	Qué debe contener
README	El punto de entrada: ¿qué hace este programa? ¿cómo lo instalo? ¿cómo lo ejecuto? ¿cómo lo pruebo?
Comentarios en el código	No el QUÉ (los nombres ya lo dicen) sino el PORQUÉ: decisiones y restricciones. Ej.: "// precio sin IVA porque el descuento se aplica antes de impuestos (requisito RF-14)"
Registro de decisiones	Qué se eligió y por qué, con fecha y alternativas descartadas. Ej.: "PostgreSQL — necesitamos transacciones ACID y el equipo conoce SQL. Descartados: MongoDB (sin experiencia), SQLite (no escala)"
Guía de contribución	Si trabaja más de una persona: cómo hacer cambios, nombrar ramas y pasar las pruebas antes de entregar

Regla del nuevo compañero: ¿podría alguien que empieza HOY poner el proyecto en marcha en menos de una hora siguiendo solo lo escrito? Si la respuesta es no, falta documentación.

Resumen: entradas → salidas por fase

Fase	Entradas	Salidas
4.1 Análisis	Objetivos, restricciones, necesidades de los usuarios	Requisitos priorizados, historias con criterios, bocetos
4.2 Diseño	Requisitos, restricciones, objetivos de calidad	Diagramas UML, decisiones de arquitectura, bocetos acordados
4.3 Implementación	Requisitos claros, diseño acordado	Código legible y ejecutable, instrucciones de arranque
4.4 Pruebas	Criterios de aceptación, código, datos de prueba	Resultados, bugs detectados y corregidos
4.5 Despliegue	Versión aprobada, configuración del entorno	Versión publicada, lista de cambios, plan de vuelta atrás
4.6 Operación	Versión desplegada, métricas iniciales	Alertas, panel de monitorización, mejoras y parches
4.7 Documentación	Código y decisiones actuales	README, comentarios, registro de decisiones

4.8 Modelos de ciclo de vida (I)

Las fases siempre están presentes. Lo que cambia según el modelo es EL ORDEN en que se recorren, CUÁNDO se entrega algo al usuario y CÓMO se gestionan los cambios.

Cascada — las fases en orden, sin volver atrás

Análisis → Diseño → Implementación → Pruebas → Despliegue. Funciona con requisitos muy estables, contratos cerrados y entornos regulados que exigen documentar cada paso. Riesgo principal: si el cliente cambia de idea a mitad, el coste de corregir es muy alto porque ya se construyó sobre una base incorrecta.

Modelo en V — cada fase de definición tiene su fase de pruebas correspondiente

El lado izquierdo define (requisitos → diseño de sistema → diseño detallado); el derecho verifica (pruebas de aceptación ← integración ← unitarias). Cada nivel de definición se comprueba con su nivel de prueba. Se usa en sectores donde el fallo tiene consecuencias graves (automoción, sanidad, aeroespacial) y se exige trazabilidad entre requisitos y pruebas.

Espiral — el análisis de riesgos en el centro de cada vuelta

En cada iteración: definir objetivos → identificar y mitigar riesgos → desarrollar y validar → planificar la siguiente vuelta. Para proyectos innovadores con mucha incertidumbre técnica. Más complejo de gestionar.

4.8 Modelos de ciclo de vida (II) — incremental, iterativo y comparativa

	Incremental	Iterativo
Qué se repite	Se añaden funcionalidades NUEVAS en cada ciclo	Se MEJORA la misma funcionalidad en cada ciclo
Analogía	Construir la casa habitación a habitación	Boceto → plano → plano definitivo
Resultado de cada ciclo	Una pieza nueva que suma al producto anterior	Una versión mejorada de lo que ya había
Cuándo usarlo	Proyecto grande divisible en funcionalidades que entregan valor por separado	Requisitos difusos que se descubren cuando el usuario ve el producto

Comparativa de todos los modelos			
Modelo	Ritmo de entrega	Gestión del cambio	Mejor para
Cascada	Una entrega al final	Costoso, hay que rediseñar	Requisitos fijos, contratos cerrados
Modelo en V	Una entrega, verificación formal	Costoso	Sectores regulados, seguridad crítica
Incremental	Entregas parciales funcionales	Moderado entre incrementos	Proyectos grandes divisibles
Iterativo	Versiones sucesivas mejoradas	Bien tolerado	Requisitos difusos que se clarifican con el uso
Espiral	Ciclos con prototipos	Bien gestionado con experiencia	Alto riesgo técnico
Ágil	Entregas frecuentes (semanas)	Alta — su punto fuerte	Cliente que necesita feedback rápido